

СУБД

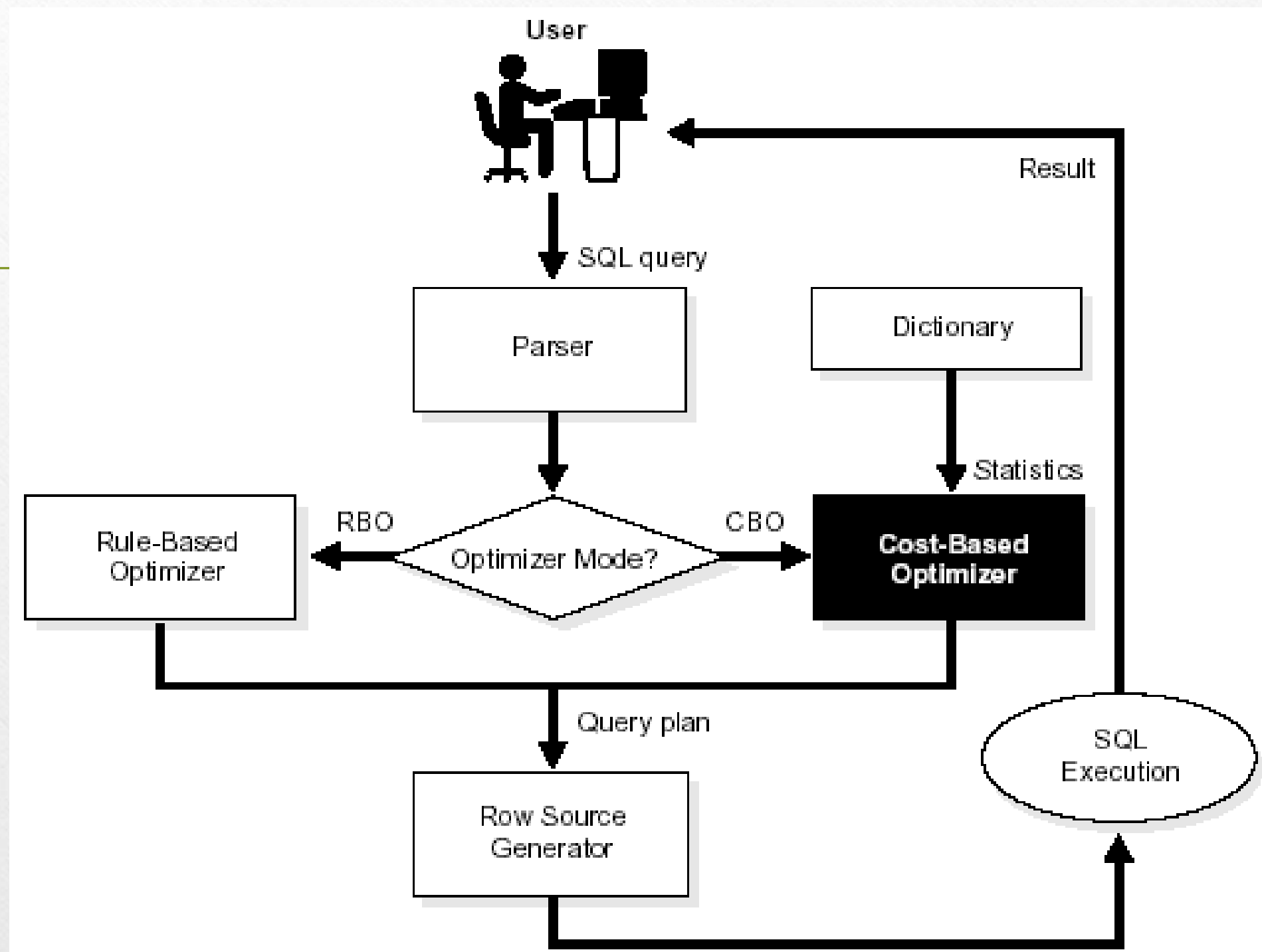
Лекция 10

Выполнение запроса (SQL-92)

1. Выполняется FROM - формируется рабочая таблица
2. к этому результату применяются предикаты предложения WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY (формально не входит в определение SELECT)

Выполнение запроса

1. Лексический анализ запроса
2. Логическая оптимизация
3. Определение планов запроса
4. Выбор оптимального плана запроса
5. Подстановка параметров
6. Непосредственное выполнение



Функции оптимизатора

1. *Вычисление выражений* и операций
2. Преобразование *SQL операторов*
3. Выбор способа *оптимизации* - по стоимости или по правилам
4. Выбор *путей доступа*
5. Выбор порядка соединений таблиц
6. Выбор метода соединений таблиц
7. Определение наиболее эффективного плана выполнения

Режимы работы оптимизатора

- RULE BASED
- COST BASED

Cost Based Optimisation

- optimizer_mode = *rule*
- optimizer_mode = all_rows
- optimizer_mode = first_rows
- optimizer_mode = *choose*
- optimizer_mode = first_rows_n

Причины ресурсоемкости запроса

- **плохая статистика** по таблицам и индексам запроса;
- **проблемы с индексами** в запросе;
- **проблемы с хинтами** в запросе;
- **неэффективно построенный запрос**;
- **неправильно настроены параметры инициализации базы данных**, отвечающие за производительность запросов.

Неэффективная статистика.

- Устаревшая статистика.
- Отсутствие статистики хотя бы в одной из таблиц, входящих во фразу From

-
- analyze table t compute statistics for table for all indexes for all columns
 - analyze table t compute statistics for table for all indexes for all indexed columns;
 - analyze table t compute statistics for table for columns i, s;
 - analyze index t_i compute statistics for table for all indexes for all columns;

-
- `execute dbms_stats.gather_table_stats
(ownname=>'stud', tabname=>'t', partname=>null,
estimate_percent=>50, block_sample=>false,
method_opt=>'FOR ALL COLUMNS',
degree=>null, cascade=>true, stattab=>null,
statid=>null, statown=>null);`

Статистика по таблицам

- Представление словаря USER_TABLES.
 - количество строк
 - количество блоков
 - количество пустых блоков
 - среднее доступное *свободное пространство*
 - количество мигрировавших строк
 - средняя *длина строки*

Статистика по индексам

- Представление словаря USER_INDEXES.
 - глубина индекса
 - КОЛ-ВО ЛИСТОВЫХ БЛОКОВ
 - КОЛ-ВО РАЗЛИЧН. *ключей*
 - средн. КОЛ-ВО ЛИСТ. БЛОКОВ НА *ключ*
 - средн. КОЛ-ВО *блоков данных* НА *ключ*
 - КОЛ-ВО УЗЛОВ ИНДЕКСА
 - *фактор кластеризации*

Статистика по столбцам

- Представления словаря USER_TAB_COLUMNS, USER_TAB_COLUMNS_STATISTICS
 - количество различных значений
 - минимальное значение
 - максимальное значение
 - КОЛ-ВО *null*

Проблемы с индексами

- **Отсутствие нужного индекса**
- **Индекс имеется, но он неэффективно построен**
 - **Малая селективность столбца, на котором построен индекс**
 - **Столбец селективный, но он входит в составной индекс**
- **Построен эффективный индекс, но он работает не эффективно**
 - **Индекс заблокирован от использования**
 - **Не собрана или неактуальная статистика по индексу**
 - **Имеется хинт, блокирующий работу индекса**
- **Имеются сильные индексы, но они соперничают между собой.**
- **Индекс давно не перестраивался**

Проблемы с хинтами в запросе

- Неэффективный хинт
- В запросе отсутствует хинт

Подсказки оптимизатору

•“О синтаксисе: `/*+ HINT HINT ... */` пробел между “+” и первой буквой подсказки имеет значение, в случае отсутствия пробела подсказка может быть игнорирована, т.е. `/*+ ALL_ROWS */` правильное использование, а `/*+ALL_ROWS */` – неправильное

•Подсказки всегда “форсируют” использование стоимостной оптимизации (cost based optimizer).

•Если в запросе используются псевдонимы (table alias), в подсказках также должны использоваться псевдонимы вместо названий таблиц:

- SQL> SELECT `/*+ FULL ( emp ) */ empno` FROM emp myalias WHERE empno > 10; -- НЕправильно
- SQL> SELECT `/*+ FULL ( myalias ) */ empno` FROM emp myalias WHERE empno > 10; -- правильно

•В подсказках не должно быть указания названия схемы:

•SQL> SELECT `/*+ index(scott.emp emp1) */...`

•Некорректные подсказки (invalid hints) игнорируются без предупреждений

Подсказки, определяющие общие цели и подходы для оптимизации плана выполнения запроса, включая правила и методы доступа к данным.

Hint

`/*+ ALL_ROWS */`

Описание

определяет целью скорейшее выполнение всего запроса с минимальным расходом ресурсов (**best throughput** при извлечении всего результирующего набора данных).

`/*+ FIRST_ROWS */`

без аргументов, предназначенная для оптимизации плана выполнения с целью скорейшего возвращения первой строки запроса

Hint

Описание

`/*+ FIRST_ROWS(n) */`

Оптимизация, основанная на стоимости (**Cost Based Optimization**) + использование правил (предпочтений в выборе плана) с целью получения лучшего времени отклика для получения первых n строк.

**`/*+ RESULT_CACHE
*/`**

Использование кэша результатов

/*+ INDEX_JOIN (
[@query_block] tablespes
[indexspec],...) */

использовать для получения результатов запроса временный индекс, получающийся в результате объединения существующих индексов методом Index [Hash] Join

/*+ MATERIALIZE */

подсказка указывает оптимизатору Oracle “материализовать” блок запроса (**query block**), в котором присутствует подсказка, в виде временной таблицы [только] на время выполнения запроса

/*+ INLINE */

Противоположна по смыслу предыдущей подсказке **/*+ MATERIALIZE*/**

```
/*+ USE_CONCAT  
([@query_block] [  
OR_PREDICATES(1) [  
PREDICATE_REORDERS(  
0)] ] ) */
```

инструктирует оптимизатор преобразовывать дизъюнктивные запросы (OR-conditions in the WHERE clause) в объединение запросов с использованием оператора **UNION ALL (CONCATENATION)** [т.е. выполнять преобразование **OR-Expansion**]. В отсутствии подсказки трансформация выполняется на основе стоимостного подхода

```
/*+ NO_EXPAND (  
[@query_block] ) */
```

Подсказка противоположная хинту **USE_CONCAT**: сигнализирует оптимизатору не использовать преобразование **OR-expansion** для запросов, содержащих дизъюнктивные условия OR или IN-lists в секции WHERE, несмотря на стоимость

Hint

Описание

**/*+ LEADING([@query_block]
[tablespec],[tablespec],...) */**

указывает оптимизатору
использовать перечисленный
порядок доступа к таблицам при
построении плана выполнения
запроса

/*+ ORDERED */

указывает проводить соединение
таблиц в том же порядке, в котором
таблицы перечислены в
конструкции FROM.

**/*+ USE_HASH([@query_block]
[tablespec] [tablespec]...) */**
**/*+ NO_USE_HASH(
[@query_block] [tablespec]
[tablespec]...) */**

указывает оптимизатору
использовать / не использовать
операцию **hash join** для соединения
каждой указанной таблицы с
прочими источниками данных

CACHE

Используйте подсказку CACHE для размещения всей таблицы в буферном кэше. Таблица размещается в наиболее часто используемом конце буферного кэша. Эта подсказка применима для небольших таблиц, к которым часто обращаются.

```
select /*+ CACHE(emp) */ ename,dname  
from emp, dept  
where emp.deptno = dept.deptno
```

FULL

Используйте подсказку FULL для выполнения полного просмотра таблицы.

```
select /*+ FULL(emp) */ ename,dname  
from emp, dept  
where emp.deptno = dept.deptno
```

INDEX

Используйте подсказку INDEX в качестве указания для Oracle по применению одного из индексов, определенных в качестве параметров.

```
select /*+ INDEX(emp,PK_EMP) */ ename,dname  
from emp, dept  
where emp.deptno = dept.deptno
```

INDEX_ASC

(по умолчанию) указывает просмотреть индекс в порядке возрастания

INDEX_DESC

указывает просмотреть индекс в порядке убывания

INDEX_COMBINE

INDEX_COMBINE вынуждает оптимизатор использовать битовые индексы.

```
select /*+ INDEX_COMBINE(emp,ENAME) */  
ename,dname  
from emp, dept  
where emp.deptno = dept.deptno
```

Используйте подсказку NOCACHE для размещения блоков таблицы в начале буферного кэша, чтобы они были меньше подвержены действию алгоритма вытеснения по давности использования.

NOCACHE

```
select /*+ NOCACHE(emp) */ ename,dname  
from emp, dept  
where emp.deptno = dept.deptno
```

Используйте подсказку PARALLEL, чтобы затребовать несколько серверных процессов для одновременного обслуживания операций на указанной таблице.

PARALLEL

```
select /*+ PARALLEL(emp) */ ename,dname  
from emp, dept  
where emp.deptno = dept.deptno
```

ROWID

Используйте подсказку ROWID для доступа к таблице по ROWID.

```
select /*+ ROWID(emp) */ ename,dname  
from emp, dept  
where emp.deptno = dept.deptno
```

USE_NL

Используйте подсказку USE_NL для выполнения соединения вложенными циклами

```
select /*+ USE_NL(emp,dept) */ ename,dname  
from emp, dept  
where emp.deptno = dept.deptno
```

Хорошие результаты дает комбинированное употребление хинтов **ORDERED**, **USE_NL** и, при необходимости, **INDEX**

Неэффективно написанный запрос.

- неэффективное соединение таблиц;
- использование NOT и NOT IN в условии where;
- блокировка индекса в силу использования неправильных функций к столбцу, по которому построен индекс;
- большая вложенность запроса или большая его длина;
- большой объем выбираемых данных, требующих подключения в работу дисков, в том числе для выполнения агрегированных функций (order by, group by и т.д.);
- неэффективные хранимые процедуры, используемые в запросе

Получение плана запросов

- EXPLAIN PLAN [SET STATEMENT_ID = 'text']
[INTO [schema .] table [@ dblink]] FOR statement;
- Select * from [schema .] table [@ dblink]
- select * from table(dbms_xplan.display);

EXPLAIN PLAN FOR

SELECT ofr.*, pr.*, gd.*

FROM sigma.product pr, sigma.goods gd, sigma.offer ofr

WHERE ofr.offer_id = pr.offer_id

AND pr.goods_id = gd.goods_id

AND gd.type_id = 2

-- получение плана запроса --

SELECT * FROM TABLE(dbms_xplan.display)

Plan hash value: 2039775141

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		81472	21M	6558	(1)	00:01:24
* 1	HASH JOIN		81472	21M	6558	(1)	00:01:24
2	TABLE ACCESS FULL	OFFER	8637	455K	37	(0)	00:00:01
* 3	HASH JOIN		81472	16M	6520	(1)	00:01:24
4	TABLE ACCESS BY INDEX ROWID	GOODS	9342	1167K	605	(0)	00:00:08
* 5	INDEX RANGE SCAN	XIE1GOODS	9342		40	(0)	00:00:01
6	TABLE ACCESS FULL	PRODUCT	1542K	132M	5909	(1)	00:01:16

Predicate Information (identified by operation id):

- 1 - access ("OFR"."OFFER_ID"="PR"."OFFER_ID")
- 3 - access ("PR"."GOODS_ID"="GD"."GOODS_ID")
- 5 - access ("GD"."TYPE_ID"=2)

-
- TABLE ACCESS FULL — сервер просмотрит все записи таблицы.
 - TABLE ACCESS BY INDEX ROWID — таблица будет просмотрена частично с помощью индекса.
 - INDEX RANGE SCAN — для получения выборки нужных значений фильтра будет использован индекс таблицы.
 - HASH JOIN — для получения выборки нужных значений фильтра будет построена хэш-таблица.
 - NESTED LOOPS — нужные значения фильтра будут получены путем полного просмотра основной таблицы и поиском записей во вспомогательной.
 - SORT MERGE JOIN — используется для соединения записей нескольких независимых источников. Сначала оба источника сортируются по объединяющему ключу, а затем происходит их слияние.
 - BUFFER SORT — в некоторых случаях Oracle может определить, что при выполнении запроса обращение к некоторому блоку данных может быть выполнено несколько раз, в этом случае Oracle помещает этот блок в специальную область, чтобы ускорить к нему доступ.
 - MERGE JOIN CARTESIAN — для получения выборки нужных значений фильтра будет организовано перемножение записей в двух таблицах

Столбец	Описание
Statement_id	Имя плана
Operation	Выполняемая операция
Object_name	Объект базы, упоминаемый в операции
Object_type	Атрибут объекта
Id	Номер приписываемый операции
Parent_id	Номер родителя
Position	Порядок обработки шагов с одинаковым Parent_id
cost	Стоимость
CPU_cost	Стоимость для процессора
IO_Cost	Стоимость операций ввода-вывода